Memory Footprints from Resident Memory and Allocator Statistics: A Footprint Model at the Cost of a Native Run

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A memory-aware scheduler needs to know how much memory a job uses over its run. MemPrint, a Pin-based footprint profiler, measures the pages and bytes a program touches, but its windowed mode costs 12.0× a native run at the median of 22 programs and 18.7× to 93.0× on programs that allocate and free memory often. The resident set size (RSS) of a native run costs nothing extra and matches the pages a program touches on array programs, but it overestimates allocator-heavy programs by a median of 17% over the whole run, and by 99% on miniVite. We show that the gap has two sources that a program can report about itself cheaply. First, the allocator keeps freed memory resident, and glibc reports the amount through `mallinfo`. Second, RSS counts the pages of shared libraries, which do not grow with the input and are shared with other processes. We propose a footprint model that samples, from one thread inside the process every 5 ms, the anonymous RSS, the proportional set size (PSS) and glibc's free bytes, and that estimates either the program's data footprint (anonymous RSS minus free bytes, plus the runtime's own pages) or its charged footprint (PSS minus free bytes). On the 7 allocator-heavy programs, the data estimate reduces the median whole-run error from +17% to −4% and the median absolute error from 20% to 9%, and it reduces miniVite's error from +99% to −8%. Its accuracy matches MemPrint's (−6% signed, 12% absolute) at a median of 0.99× the run time of a plain RSS run (up to 1.21×, on darknet), against 34.4× for MemPrint. On the 16 other programs, it matches RSS. We also test the alternatives one at a time. Releasing freed memory to the kernel during the run removes less of the gap, language runtime statistics fit only two programs, and per-window referenced pages underestimate by up to 70%.

1 Introduction

A scheduler that packs jobs onto nodes by their memory use needs an estimate of each job's footprint over time, because a job's peak lasts only part of its run [2, 17, 10]. A reservation that follows the footprint over time lets a node hold more jobs than a reservation of the peak, but the estimate must not fall below what the job uses, or the job runs out of memory.

MemPrint measures a program's footprint with Pin [11]. It records the bytes and pages each program touches and the time at which each block is freed. Its windowed mode watches 5% of the run and costs 12.0× a native run at the median on the 22 programs of our corpus that it ran on. However, on the programs where a footprint estimate helps the most, those that allocate and free memory throughout their run, the cost rises to 18.7× to 93.0×, because Pin instruments every memory access and every allocator call.

The resident set size of a native run is the obvious cheap alternative. On array programs, RSS is the footprint, because these programs allocate their arrays once and touch them. On allocator-

heavy programs, RSS overestimates the footprint over the run by a median of 17%, and by 99% on miniVite [7], because the kernel counts memory that the program no longer uses.

In this work, we make the following contributions:

- We identify the two parts of RSS that are not part of the footprint on our corpus, freed memory that glibc keeps resident and the pages of shared libraries, and we measure their size per program (Section 2).
- We propose a footprint model that combines RSS, PSS and glibc's free bytes, sampled every 5 ms from a thread inside the process through an `LD_PRELOAD` library, with two estimates for two uses (Section 3).
- We show on 23 programs that the model matches MemPrint's accuracy on allocator-heavy programs and matches RSS on the others, at a median of 0.99× the run time of a plain RSS run (Section 5).
- We test four other ways to close the gap, each alone, and show why each is less accurate (Section 6).

2 Background: what RSS counts

The RSS of a process is the number of its pages in memory. An anonymous page enters RSS when the program first touches it and leaves when the program or its allocator unmaps it or releases it with `madvise`. A file-backed page, such as a page of a shared library's code, enters RSS when any process reads it and stays as long as it is mapped and the kernel does not reclaim it. The *footprint* we estimate is the set of pages the program has touched and not yet freed, plus the pages the runtime needs before the program starts. The truth in our evaluation is this footprint as MemPrint measures it (Section 4). RSS differs from it in four ways.

Allocator retention. The C library does not return every freed block to the kernel. glibc's `malloc` serves blocks below a threshold (128 KiB, raised dynamically up to 32 MiB) from its arenas, keeps freed blocks in free lists for reuse, and releases memory only from the top of the heap when the free space there exceeds the trim threshold [6]. Other allocators also release freed memory late or in batches [5, 8]. A program that frees and allocates memory throughout its run therefore holds freed pages in RSS. glibc reports the free bytes of all its arenas through `mallinfo` (the field `fordblks`).

Shared libraries. RSS counts every resident page of every mapped file, including the code of the C library, the C++ runtime and, for MPI programs, the MPI library. These pages do not depend on the input, and other processes on the node map the same pages. The PSS of a process divides each resident page by the number of processes that map it.

Language runtimes with their own allocators. CPython's object allocator takes 256 KiB arenas from the system and returns an arena only when all its objects are freed. GCC's garbage-collected heap also maps its own pages. glibc's statistics do not cover these allocators.

Page granularity. RSS counts whole pages, and a program may touch part of a page. On our corpus, the pages a program touches exceed the bytes it touches by 1% at the median, so we use the pages as the truth.

3 The footprint model

The model estimates a footprint curve from four quantities sampled at the same time inside the process. Let $A(t)$ be the anonymous RSS, $P(t)$ the PSS, $F(t)$ the bytes free in glibc's arenas, and B the runtime's own pages, the smallest RSS of an earlier run of the program, which is measured once per program. The model gives two estimates.

Estimate 1, the data footprint, is

$$\hat{f}_1(t) = \max\big(A(t) - F(t),\, 0\big) + B. \tag{1}$$

It counts the anonymous pages the allocator has not freed, and it leaves out file-backed pages. It estimates the memory that grows with the input, which is what a scheduler predicts and what our truth measures.

Estimate 2, the charged footprint, is

$$\hat{f}_2(t) = \max\big(P(t) - F(t),\, 0\big). \tag{2}$$

PSS is the anonymous RSS plus each file-backed page divided by the number of processes that map it, so the estimate charges the job for its share of shared libraries and for all of a library that only it maps. It estimates the memory the job adds to a node.

Both estimates subtract free bytes from resident memory rather than count live allocations. A counter of live `malloc` blocks misses every allocation that does not go through `malloc`, so an estimate built from it falls below the truth whenever a program uses its own allocator. Subtracting free bytes errs the other way. Memory that glibc does not know about stays in RSS, so a program whose allocator glibc cannot see gets the RSS estimate, which is too high but safe.

Implementation. A shared library of 100 lines of C, loaded with `LD_PRELOAD`, starts one thread in its constructor. Every 5 ms the thread reads `/proc/self/smaps_rollup` for the RSS, PSS and anonymous RSS, calls `mallinfo`, and writes one line to a buffered file. The thread does not stop the program's threads, and it takes glibc's arena locks only for the duration of `mallinfo`. The machine runs glibc 2.28, which has no `mallinfo2`, so the free bytes are a 32-bit field that wraps above 4 GiB. A newer glibc reports them in 64 bits. The library removes itself from `LD_PRELOAD` so that child processes are not sampled, and a program made of several processes would need one output file per process.

4 Methodology

Machine. All runs use a two-socket Intel Xeon E5-2620 v4 (16 cores, 32 threads) with 62 GiB of memory, Linux 4.18 and glibc 2.28. Runs execute one at a time on an otherwise idle machine.

Corpus. We use the 23 programs of the corpus for which a pages-touched truth exists, each at its largest input: PolyBench (2mm, atax, gemm, jacobi-2d), the NAS parallel benchmarks (cg, is, mg), the GAP benchmark suite (bfs, cc, pr, sssp) [3], the proxy applications HPCCG, Kripke, LULESH, miniFE, miniVite and Quicksilver, darknet AlexNet inference, GCC's `cc1` compiling SQLite, and Lua [9], CPython, Perl and SQLite running table, JSON, hash and SQL workloads. We call 7 programs *allocator-heavy*, because they allocate and free memory throughout their run: miniVite, CPython, Lua, SQLite, Perl, Quicksilver and `cc1`. We chose the set before these experiments, from the earlier observation that RSS and MemPrint disagree on these programs and agree on the others. The runs last 0.1 to 12 s and peak at 27 to 273 MiB of RSS.

Truth. The truth is the number of 4 KiB pages the program touches and has not freed, which Pin measures in MemPrint's splitter mode with page counting and free tracking, plus the runtime's

Table 1: Medians over programs, each the median of 3 runs. Whole-run errors and peak errors are percentages of the truth (pages touched plus the runtime's pages). The 90th percentile of the absolute error and the lowest peak error show the worst cases. Overhead is the run time over a plain RSS run (×).

			Whole-run error (%)			Peak error (%)		Overhead
Programs	Estimate	n	signed	absolute	q90 abs.	median	lowest	(×)
Allocator-heavy (7)	RSS	7	+17	20	57	+10	0	1.00
	Data (1)	7	−4	9	13	−2	−4	0.99
	Shared (2)	7	+7	11	23	−1	−3	0.99
	MemPrint	7	−6	12	20	−5	−16	34.4
Others (16)	RSS	16	0	8	36	0	−4	1.00
	Data (1)	16	−1	12	34	−1	−5	0.98
	Shared (2)	16	−4	14	32	−4	−9	0.98
	MemPrint	15	−1	10	32	−1	−2	8.9
All (23)	RSS	23	+7	16	40	+1	−4	1.00
	Data (1)	23	−1	11	28	−1	−5	0.99
	Shared (2)	23	0	12	35	−3	−9	0.99
	MemPrint	22	−1	11	25	−1	−16	12.0

pages B. The truth's time axis is Pin's count of memory references. We divide each run into 100 equal steps of progress and take the largest value of each curve in each step. The estimates are placed on the same 100 steps by the wall clock of their own run.

Metrics. We report three errors of an estimate e against the truth t over the 100 steps. The signed whole-run error is $(\sum e - \sum t)/\sum t$, which is positive when the estimate reserves more memory over the run than the program uses. The absolute whole-run error is $\sum |e - t|/\sum t$. The peak error is $\max e/\max t - 1$. Each program runs 3 times, and we report the median of each error over the runs. The overhead of an estimate is the median wall time of its run over the median wall time of a run under the plain RSS sampler of the corpus (`rss_sampler.py`, an external process that reads `smaps_rollup` every 5 ms).

Baselines. *RSS* is the RSS curve of the plain sampler's run. *MemPrint* is the windowed mode that watches 5% of the run (W5), placed on the native wall clock by the hardware counts of loads and stores of a native run, plus B. Its cost is the wall time of the W5 run and of the counter run over the wall time of an earlier plain RSS run. MemPrint has no W5 curve for darknet, so its rows cover 22 programs.

5 Results

Accuracy on allocator-heavy programs. The data estimate reduces the median whole-run error of RSS from +17% to −4% and the median absolute error from 20% to 9% (Table 1). Its accuracy matches MemPrint's, which has a median signed error of −6% and a median absolute error of 12%. Per program (Table 2), the data estimate reduces miniVite's error from +99% to −8%, CPython's from +30% to +1%, and Lua's from +23% to +1%, and its absolute error is at most 14% on every allocator-heavy program. Its peak is within 4% of the truth's on all 7 programs, with a median of −2%, while the RSS peak is 10% too high at the median and 58% too high on miniVite.

The charged estimate. The charged estimate lies above the truth on 5 of the 7 programs, with a median signed error of +7%, because it keeps the pages of libraries that only this job maps.

Table 2: Whole-run error per program (% of the truth), signed and absolute, median of 3 runs. Overhead is the run time over a plain RSS run (×). The model's two estimates come from the same run. MemPrint has no W5 run of darknet.

	RSS		Data (1)		Shared (2)		MemPrint		Overhead (×)	
Program	sign.	abs.	sign.	abs.	sign.	abs.	sign.	abs.	model	MemPrint
Allocator-heavy										
miniVite	+99	99	−8	11	+36	37	−20	21	0.99	40.2
python	+30	30	+1	9	+4	8	+1	6	1.00	27.3
lua	+23	26	+1	9	−2	11	−14	20	0.98	23.6
sqlite	+17	20	+8	14	+7	14	+9	19	0.96	50.3
perl	+16	16	−5	12	−4	12	−9	12	0.99	34.4
quicksilver	+15	17	−4	4	+7	9	−5	5	0.99	18.7
cc1	+12	12	−7	7	+10	10	−6	6	1.00	93.0
Others										
2mm	−1	2	0	1	−4	4	−1	1	1.01	5.9
atax	−21	21	−20	20	−25	25	−16	16	0.60	23.1
darknet	−5	6	−13	13	−14	14	–	–	1.21	–
gap-bfs	+35	43	+28	38	+23	36	+21	37	0.98	8.9
gap-cc	+30	43	+23	39	+18	37	+20	38	0.96	8.5
gap-pr	+21	30	+15	25	+10	25	+12	26	0.97	7.4
gap-sssp	+25	27	+26	29	+24	28	+17	20	1.04	8.3
gemm	−1	1	−1	1	−5	5	−1	1	1.02	5.6
hpccg	−1	2	−2	2	−3	3	−1	1	0.98	12.2
jacobi-2d	−2	2	−1	1	−6	6	−1	1	1.00	4.4
kripke	+2	3	+1	3	+1	2	0	3	1.01	11.5
lulesh	+7	12	+5	15	+3	15	0	16	1.00	11.3
minife	+2	4	−2	2	0	2	−2	2	0.99	13.9
npb-cg	0	2	−5	5	−10	10	−3	3	0.96	11.9
npb-is	−17	17	−18	18	−21	21	−18	18	0.98	5.6
npb-mg	−10	10	−11	11	−14	14	−10	10	0.84	22.4

On miniVite, these pages belong to the MPI and PMIx libraries (Section 5.1), and the estimate is +36%. Unlike the data estimate, the charged estimate counts memory that the truth does not count, so its error against the truth does not measure its accuracy as an estimate of a node's memory use.

Programs that RSS already estimates. On the 16 other programs, the model's estimates match RSS. Their median signed errors are −1% (data) and −4% (charged), against 0% for RSS (Table 1). The median absolute error is 12% for the data estimate and 8% for RSS, but this difference comes from run-to-run variation and not from the model. The RSS that the model's own runs record has a median absolute error of 11.5% on the same programs, and the data estimate of the same runs has 12%.

Errors that every estimate shares. On 7 of the 16 other programs, every estimate, MemPrint included, has a whole-run error of 10% or more in the same direction. The GAP programs are overestimated by +10% to +35%, and atax, NPB-IS and NPB-MG are underestimated by −10% to −25%. The peak errors of the same estimates are within 6%, so the estimates and the truth agree on the amount of memory and disagree on when it is used. The truth is on Pin's count of memory references, and the estimates are on the wall clock, so a phase that executes few memory references per second, such as GAP's graph construction, takes a larger share of the run on the

Table 3: Parts of the model's run, each summed over the run as a percentage of the truth's sum (the truth is 100). Data is estimate 1. Free is glibc's free bytes. File RSS and file PSS are the RSS and PSS of file-backed pages (RSS minus anonymous RSS, and PSS minus anonymous RSS). Median of 3 runs.

Program	Data (%)	Free (%)	File RSS (%)	File PSS (%)
miniVite	92	38	87	56
python	101	15	21	9
lua	101	23	7	1
sqlite	108	7	9	4
perl	95	15	10	4
quicksilver	96	2	23	15
cc1	93	2	23	20
others (median of 16)	99	1	5	0

wall clock. The errors therefore measure the difference between the two clocks, and they bound the accuracy any wall-clock estimate can show against this truth.

Overhead. The model's run takes 0.99× the time of a plain RSS run at the median, and 0.96× to 1.04× on 20 of the 23 programs. The plain RSS run includes the 40 ms that the external sampler takes to start, which explains the ratios of 0.60× for atax and 0.84× for NPB-MG, whose runs last 0.10 s and 0.31 s. darknet runs 1.21× slower under the model in all 3 runs. A thread that sleeps for 0.99 s between samples slows darknet by the same 1.22×, so the cost comes from the presence of a second thread in a single-threaded program, and not from the sampling. We did not identify which code path the second thread slows down. MemPrint's W5 run with its counter run takes 12.0× a native run at the median, 8.9× on the other programs and 34.4× on the allocator-heavy programs, with 93.0× on `cc1`.

5.1 Where the remaining RSS goes

Table 3 shows the size of each part of the model's run against the truth. On the allocator-heavy programs, glibc holds free memory equal to 7% to 38% of the truth on 5 of the 7 programs, and file-backed pages make up 7% to 87% of the truth. On the other programs, both parts are small, at a median of 1% and 5%.

miniVite has the largest share of both parts. glibc's free bytes amount to 38% of the truth, and file-backed pages amount to 87%. A sample of `/proc/pid/smaps` in the middle of a run shows where the file-backed pages are. The MPI library holds 2348 KiB, PMIx 2096 KiB, the C library 1496 KiB and the C++ runtime 1356 KiB, against a truth of 9.7 MiB on average over the run. The PSS of the C library is 77 KiB and that of the C++ runtime 199 KiB, because other processes map the same pages, but the PSS of the MPI and PMIx libraries equals their RSS, because only miniVite maps them. Subtracting glibc's free bytes alone therefore reduces miniVite's error only from +99% to +64% (Section 6), and removing the file-backed pages reduces it to −8%.

6 Other ways to close the gap

Before we settled on the model, we tested four alternatives on the allocator-heavy programs, each on its own (Table 4). The first, RSS minus glibc's free bytes, is the first half of the model.

Releasing freed memory during the run. Instead of subtracting free bytes, the program can return them to the kernel, so that RSS drops when the program frees memory. We tested two ways. The first calls `malloc_trim(0)` every 5 ms from a thread in the process, which releases the

Table 4: Signed whole-run error (% of the truth) of each alternative used alone on the allocator-heavy programs, median of 3 runs, and its overhead over a plain RSS run (×). Runtime statistics exist only for CPython and Lua. These runs keep the file-backed pages in every estimate.

Program	RSS	RSS−free	trim 5 ms	trim thr. 0	runtime stats	referenced
miniVite	+103	+64	+56	+53	–	−70
python	+30	+14	+15	+16	+9	+15
lua	+23	+2	+5	+19	−18	+4
sqlite	+17	+12	+15	+18	–	−18
perl	+16	+1	+10	+12	–	−12
quicksilver	+16	+16	+13	+13	–	−56
cc1	+14	+12	+12	+13	–	−69
median signed	+17	+12	+13	+16	−4	−18
median absolute	20	14	15	17	38	32
overhead (×)	1.00	0.99	1.03	1.02	0.96	1.02

free pages inside every arena with `madvise`. The second sets glibc's trim threshold to 0, which releases free memory at the top of the heap after every `free`. Both reduce the median error less than subtracting free bytes, +13% and +16% against +12%. A trim threshold of 0 releases only the top of the heap, which explains why it changes Lua's and SQLite's errors by less than 4 points. Both alternatives also change the program's allocation behaviour, and the trim thread costs up to 1.10× on Perl.

Language runtime statistics. For CPython, we sampled the free bytes of its object allocator (the arena bytes minus the bytes in allocated blocks, from `sys._debugmallocstats`) from a thread started in `sitecustomize`. For Lua, we sampled the heap size the garbage collector reports (`collectgarbage("count")`) from a debug hook every 10^6 virtual machine instructions. CPython's whole-run signed error drops to +9%, but its absolute error rises to 44%, which means that the estimate is too low in some parts of the run and too high in others. We did not determine why. Lua's heap count leaves out the interpreter's own memory and gives −18%. The statistics exist only for these two programs, and GCC reports its heap only at the end of a compilation.

Referenced pages per window. The kernel can clear the referenced bits of a process's pages (`/proc/pid/clear_refs`), after which `smaps_rollup` counts the pages the process has touched since. We cleared the bits 20 times per run and took the pages referenced in each window as the footprint of that window, in the manner of a working set [4]. The estimate falls below the truth by up to 70% (miniVite), 69% (`cc1`) and 56% (Quicksilver), because a page that the program holds but does not touch within a window is not counted. A scheduler that reserved this estimate would run these programs out of memory.

7 Limitations

A limitation of the free-byte correction is that it reads only glibc's statistics, which leaves the memory of other allocators in the estimate. A program linked with jemalloc or TCMalloc needs that allocator's own statistics [5, 8], and the arenas of CPython's object allocator and GCC's garbage-collected heap stay in the estimate as RSS. On our corpus, the data estimate is nevertheless within 7% of the truth on CPython and `cc1`, and it is unclear whether this result holds for inputs on which these allocators retain more memory.

A limitation of the data estimate is that it leaves out all file-backed pages, which is wrong for a

program whose data is a mapped file. A language model server that maps its weights, for example, holds most of its footprint in file-backed pages. The charged estimate counts these pages, and we leave a rule that separates a program's data files from its libraries for future work.

A limitation of the free-byte correction is also that some free bytes may not be resident, such as the untouched part of the top of the heap, which makes the data estimate lower than the truth. The data estimate is below the truth on 4 of the 7 allocator-heavy programs, by -4% to -8% over the run, and its peak is at most 4% low, so a scheduler that uses it needs a headroom of a few percent.

A limitation of the sampling thread is that it adds a second thread to a single-threaded program, which slows darknet by $1.21\times$ (Section 5). Sampling from an external process avoids the thread, but glibc's statistics can be read only inside the process, and we leave a design without a thread for future work.

The evaluation covers a single machine, short runs of 0.1 to 12 s, single-process programs, and estimates of the same input as the run. It does not test whether the model's curve from a small input transfers to a larger input, or how a scheduler performs with the model's estimates, and we leave both for future work. The truth's clock of memory references also limits the accuracy that any wall-clock estimate can show (Section 5).

8 Related work

Footprint measurement. Valgrind's Massif and DHAT [13, 12] and MemPrint measure the heap or the touched memory of every allocation by instrumenting the program. Our model reads counters that the kernel and glibc already keep. Unlike MemPrint, it does not attribute memory to allocation sites or show which bytes of a block the program touches.

Page access sampling. DAMON [15] and Thermostat [1] sample the accessed bits of pages to find cold memory, and PEBS sampling [14] records the addresses of sampled loads. These methods measure which pages a program uses within a period of time, which is the working set [4] and not the footprint. Our referenced-page alternative (Section 6) shows that the working set of a window falls far below the footprint on allocator-heavy programs.

Memory prediction for scheduling. KS+ [2] predicts a workflow task's memory over time from the RSS of earlier runs, and Zacarias et al. [17, 18] predict the memory of HPC jobs from earlier runs. Pond [10] predicts how much of a virtual machine's memory goes untouched. These systems learn from RSS or from allocated memory, and our model gives them a training signal without the allocator's retained memory and the job's shared libraries at the same cost as RSS.

Allocators that return memory. Mesh [16] compacts the heap to return fragmented memory to the kernel, and TCMalloc's Temeraire [8] releases free huge pages. These allocators reduce the gap between RSS and the footprint by changing the program, whereas our model measures the gap without changing the program's allocations.

9 Conclusion

RSS overestimates the footprint of programs that allocate and free memory throughout their run, because glibc keeps freed memory resident and because RSS counts shared libraries. A model that subtracts glibc's free bytes from the anonymous RSS and adds the runtime's own pages reduces the median whole-run error on these programs from $+17\%$ to -4%, which matches MemPrint's accuracy, at a median of $0.99\times$ the run time of a plain RSS run instead of $34.4\times$. On array

programs, it matches RSS. Pin-based profiling is therefore needed for attributing memory to code and for allocators that glibc cannot see, and not for the footprint curve a scheduler uses.

Artifacts. `paper_rss/tools/mprshim.c` is the sampling library, `run_model.sh` and `run_options.sh` run the experiments, and `compute.py` produces every table. The runs are in `~/memprint-data/2026-10-26-rsscurve/opts`.

References

[1] Neha Agarwal and Thomas F. Wenisch. Thermostat: Application-transparent page management for two-tiered main memory. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 631–644, 2017.

[2] Jonathan Bader, Ansgar Lößer, Lauritz Thamsen, Björn Scheuermann, and Odej Kao. KS+: Predicting workflow task memory usage over time. In *IEEE 20th International Conference on e-Science*, 2024.

[3] Scott Beamer, Krste Asanović, and David Patterson. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015.

[4] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[5] Jason Evans. A scalable concurrent malloc(3) implementation for FreeBSD. In *BSDCan*, 2006.

[6] Free Software Foundation. *The GNU C Library Reference Manual*, 2024. Section "Memory Allocation Tunables" (`M_MMAP_THRESHOLD`), `https://www.gnu.org/software/libc/manual/`.

[7] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[8] A. H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. Beyond malloc efficiency to fleet efficiency: A hugepage-aware memory allocator. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 257–273, 2021.

[9] Roberto Ierusalimschy, Luiz Henrique de Figueiredo, and Waldemar Celes. Lua—an extensible extension language. *Software: Practice and Experience*, 26(6):635–652, 1996.

[10] Huaicheng Li et al. Pond: CXL-based memory pooling systems for cloud platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[11] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[12] Nicholas Nethercote and Julian Seward. *DHAT: a dynamic heap analysis tool*. Valgrind user manual, `https://valgrind.org/docs/manual/dh-manual.html`.

[13] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '07, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.

[14] Aleix Roca Nonell, Balazs Gerofi, Leonardo Bautista-Gomez, Dominique Martinet, Vicenç Beltran Querol, and Yutaka Ishikawa. On the applicability of pebs based online memory access tracking for heterogeneous memory management at scale. In *Proceedings of the Workshop on Memory Centric High Performance Computing*, MCHPC'18, page 50–57, New York, NY, USA, 2018. Association for Computing Machinery.

[15] SeongJae Park, Yunjae Lee, and Heon Y. Yeom. Profiling dynamic data access patterns with controlled overhead and quality. In *Proceedings of the 20th International Middleware Conference Industrial Track*, pages 1–7, 2019.

[16] Bobby Powers, David Tench, Emery D. Berger, and Andrew McGregor. Mesh: Compacting memory management for C/C++ applications. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2019.

[17] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Memory demands in disaggregated HPC: How accurate do we need to be? In *International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2021.

[18] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Dynamic memory provisioning on disaggregated HPC systems. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC-W)*, 2023.